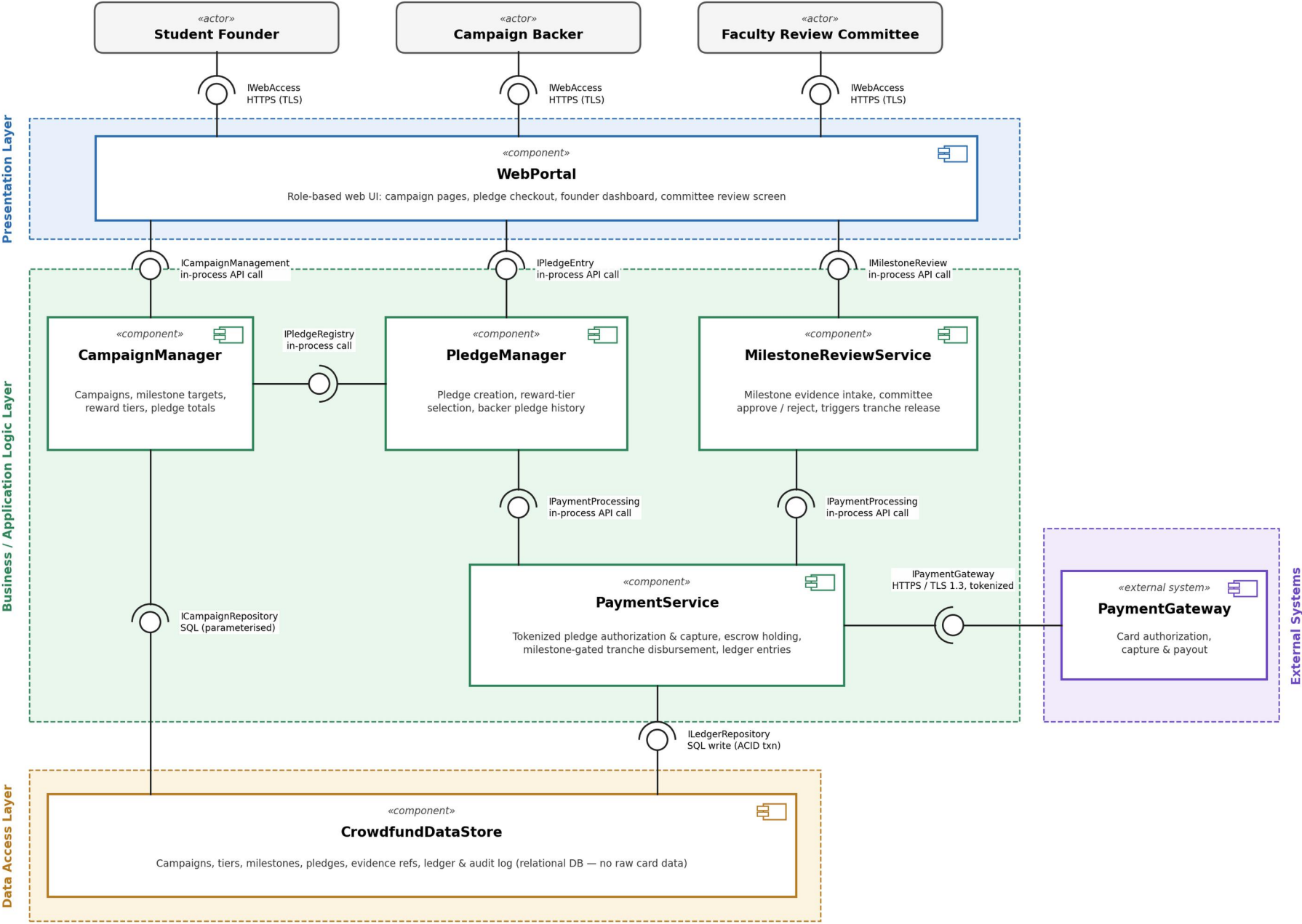


UML Component Diagram — Student Startup Crowdfunding Platform

Architectural Style: Layered (3-Tier) Architecture | Rohan Agarwal · PES1UG24AM921 · SE Lab 3 (Problem Statement #40)



Notation: ○ Provided interface (ball) (Required interface (socket) ○ Required interface (socket) ○ Ball + socket = assembly connector (label: interface / protocol)

SE Lab 3 — Component Modelling & Architectural Pattern Selection

Architectural Justification: Student Startup Crowdfunding Platform (Problem Statement #40)

Rohan Agarwal | SRN: PES1UG24AM921 | Dept. of CSE, PES University

1. Architecture Selection

We chose the **Layered (3-Tier) Architecture** for the Student Startup Crowdfunding Platform. The system has a Presentation layer (WebPortal), a Business layer (CampaignManager, PledgeManager, MilestoneReviewService, PaymentService) and a Data Access layer (CrowdfundDataStore). The external PaymentGateway is reached only through PaymentService, and all calls go through declared provided/required interfaces.

2. Two Scenario-Specific Reasons

Reason 1 — Money movement needs one transactional boundary. Pledges, escrow holding and milestone-gated tranche releases must stay consistent: a tranche must never be released without a recorded committee approval, and a pledge must never be counted twice. With layers over one relational database, each of these is an ordinary ACID transaction inside the business layer. Microservices would split campaign, pledge and ledger data across services and need sagas or distributed transactions for every pledge and release, which is hard to get right for money and unnecessary at the scale of one university's student ventures.

Reason 2 — Business rules must be isolated from the UI and the payment provider. Milestone gating, reward-tier availability and review rules live in the business layer, so the web UI can be redesigned (or a mobile app added) without touching them. Replacing the payment gateway affects only PaymentService behind IPaymentProcessing. A plain Client-Server split would not give this internal separation, and the layered design still ships as one deployable unit that a small student team can run.

3. Security Advantage

Layering confines payment-card risk and fund release to controlled paths. Card details go straight to the gateway and PaymentService stores only the returned token, so no raw card data reaches the WebPortal or the data layer; this keeps PCI-DSS scope to a single component. The only route to disburse funds is MilestoneReviewService calling IPaymentProcessing after a committee approval is recorded. WebPortal has no interface to PaymentService, so a compromised Founder or Backer session cannot trigger a disbursement. The layer boundaries are also the natural place for role checks (Founder, Backer, Committee) and audit logging.

4. Performance Benefit

Campaign pages, tier availability and pledge totals are served by in-process calls against one database, so there is no service-to-service network hop or serialisation on the browsing and pledging path. The only remote call is the gateway authorisation (IPaymentGateway), so response time is bounded by the database and that single call rather than by chained service latency. Read-heavy campaign pages can also be cached inside CampaignManager without changing any interface.

5. Trade-off Accepted

The cost is limited independent scaling: if the platform grew to many institutions or heavy campaign-launch traffic, PaymentService and the ledger would need extracting into separate services. For the stated single-university scope this is acceptable, and the interface boundaries make that extraction straightforward.